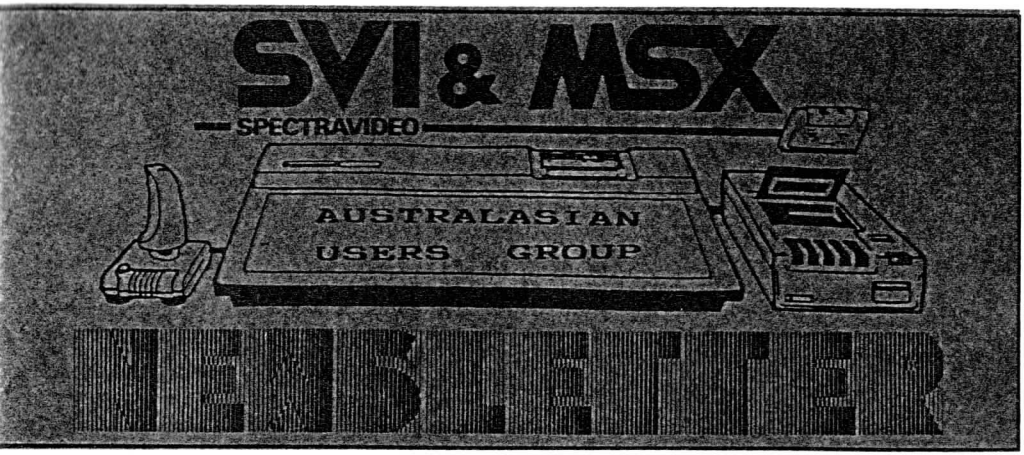

REGISTERED BY AUSTRALIA POST PUBLICATION No. TBH 0917 CATEGORY "B"

ISSUE NO.	ANNUAL SUBSCRIPTION	DATE
3 - 5/6	AUSTRALIA ..... $20.00 OVERSEAS ..... $25.00 OVERSEAS AIRMAIL ... $30.00 YEAR BOOK 83/84 .... *$20.00*	*FEB-MAR 1985*

CONTENTS

NEWSLETTER CORRESPONDENCE

S.A.U.G.,
P.O. BOX 191,
LAUNCESTON SOUTH,
TASMANIA, 7249.

(003) 442493

LIBRARY CORRESPONDENCE

S.A.U.G. LIBRARY,
1 CONRAD AVENUE,
GEORGE TOWN,
TASMANIA, 7253.

(003) 822919

M.S.X. Disk Bug Fixed !!

By. S.W. McNamee

After the editor's glowing report on the new SVI X'Press I could no longer resist the urge, so last September I proudly took posession of my new 738. Several late nights later I finally got around to playing with MSX-DOS (I am an inveterate CP/M hacker), and was unable to copy a disk without getting several disk errors. I solved that by using the CP/M utility BACKUP (Good old CP/M). Over the next few days I lost several files and a couple of times even wiped out the directory track. It finally dawned on me that something was wrong with MSX-DOS. Back to Rose Music went the X'Press and after a couple of weeks they sent it back saying there was nothing wrong with it. They gave me a new master disk in case there was something wrong with the old one. You guessed it ... same problem. Back to Rose Music who sent it to Melbourne. Another couple of weeks and back came the message "Nothing wrong with it!". Well I jumped up and down in the one spot and said there is definately something wrong somewhere. Finally in Janurary I got a brand new X'Press swapped for mine and major frustration - IT had the same problem.

In the mean time I had the use of a third machine for a short while which also had the fault. Several other X'Press owners I had talked to also experienced trouble with MSX-DOS. It soon sank through the old grey matter that there was a bug somewhere in the disk control software (ROM).

Out came the trusty dissassembler (DISZILOG) and after several dozen pages of print out and 2 days of feverish activity I finally came to the conclusion that the problem was a lack of head settling time in the write sector routine. A small patch to fix this was written and I started looking for somehwere to put it. I then discovered that MSX-DOS leaves 24 bytes of RAM unused at FFD7. Wonder of wonders the patch is 23 bytes long - I seemed to be in business. I loaded it in, gingerly fired up COPY and lo and behold got a clean disk copy. To date I have been running the patch for about 6 weeks and have not had a single disk error - end of story.

There are two BASIC programs to implement the patch.

1. DSKMOD.BAS is for use when DISK BASIC is booted up without going through MSX-DOS. It should be run each time you turn on the computer.

2. DSKMOD1.BAS is a BASIC program that creates a file called DSKMOD.COM which is a command file for use when booting up MSX-DOS which MUST be run each time you boot. You only have to use this program once. When you have a copy of DSKMOD.COM you can copy it to any disks you need.

Note that if you enter DISK BASIC from MSX-DOS after running DSKMOD.COM you need not run DSKMOD.BAS.

The easiest way to implement the patch is to use DSKMOD1.BAS to create DSKMOD.COM and then make this command part of an AUTOEXEC.BAT file. The patch will then be loaded every time you boot MSX-DOS.

DSKMOD.BAS (m.s.x.)

by : S.V. Mc Namee.

This Program may be entered using the 'INPUT' program from Newsletter 2 - 2 (NOV. 84.) or The Year Book.

```
CI      10 ' ********************************
AB      20 ' **** PROGRAM: DSKMOD.BAS *****
CK      30 ' ********************************
CL      40 '
BE      50 ' AUTHOR: Steve McNamee
AB      60 ' DATE  : 18-1-86
AK      70 ' USE   : This is a patch to fix
AC      80 ' the bug in the SV-738 X-Press
FI      90 ' disc ROM which corrupts
EJ     100 'sectors.
AJ     110 '
GK     120 ' This program MUST be run every
HL     130 ' time you boot up DISC BASIC.
AG     140 '
AF     150 '
CC     160 AD=&HFFE6
BO     170 FOR N=0 TO 23
CJ     180 READ D$:D=VAL("&h"+D$)
FP     190 POKE AD+N,D
CJ     200 NEXT N
CP     210 NEW
AL     220 END
AI     230 '
AH     240 '
BC     250 DATA E5,F5,3A,B9,7F,21,FE,FF
CK     260 DATA BE,77,28,08,21,00,40,2B
AH     270 DATA 7C,B5,20,FB,F1,E1,C9,00
END
```

DSKMOD1.BAS (m.s.x.)

by : S.V. Mc Namee.

This Program may be entered using the 'INPUT' program from Newsletter 2 - 2 (NOV. 84.) or The Year Book.

```
CI      10 '*********************************
FA      20 '***** PROGRAM: DSKMOD1.BAS *****
CK      30 '*********************************
CL      40 '
BE      50 ' AUTHOR: Steve McNamee
AB      60 ' DATE  : 18-1-86
CP      70 ' USE   : This program creates a
DB      80 ' .COM file for use under MSX-DOS
AF      90 ' This command is used to load
BM     100 'the patch to fix the bug in the
BJ     110 'SV-738 X-Press disc ROM. The
DM     120 'command MUST be run every time
DO     130 'MSX-DOS is booted. The easiest
JH     140 'way to do this is to make it
AH     150 'part of an AUTOEXEC.BAT file.
BL     160 'The name of the command is
BG     170 'DSKMOD.COM
AC     180 '
AB     190 '
EH     200 OPEN "DSKMOD.COM" FOR OUTPUT AS #1
CJ     210 FOR N=0 TO 38
DA     220 READ D$:D=VAL("&h"+D$)
AF     230 PRINT #1,CHR$(D);
CN     240 NEXT N
BJ     250 CLOSE #1
DE     260 NEW
BA     270 END
BG     300 DATA 00,00,00,21,0F,01,11,E6
CI     310 DATA FF,01,17,00,ED,B0,C9
BD     350 DATA E5,F5,3A,B9,7F,21,FE,FF
CJ     360 DATA BE,77,28,08,21,00,40,2B
AG     370 DATA 7C,B5,20,FB,F1,E1,C9,00
END
```

Exploring Basic Pt-16

by L.A. Dunning

This part deals with disk drives and how to avoid making errors while using them. It also covers techniques with interrupts.

DISK ERRORS

When Disk Basic is loaded, it also adds a set of errors than can only occur when you use a disk. These are neccesary if you want to avoid losing data, files, or overwriting disks. The extra errors are:

61	Bad Allocation Table
62	Bad Drive Number
63	Bad Track/Sector
64	File still open
65	Disk not mounted
66	Deleted Record
67	File already exists
68	Disk full
69	File Write Protected
70	Disk I/O error
71	Disk offline
72	Rename accross Disk

Most of these are fairly obvious in meaning and you should have encountered them already, especialy Bad Allocation Table or File already exists. Anyone who can tell me how to get errors 65, 66 or 71 will get a certificate of observation from me. Whenever my drive doesn't have a disk in it, or the catch isn't down, it just hangs. So how do you get errors 65 or 71? There doesn't appear to be anyway to 'delete' a record. You can write over it, but not delete it.

Errors 61 to 63 are straightforward. You will get error 64 if you try to KILL a file that is currently open. Error 67 occurs when you try to rename a file and the new name is the name of an existing file. Error 80 happens when all available tracks on the disk are used. Error 69 occurs when you try to write to the disk and either the file or disk is set for Write Protect ("P"using SET).

You get error 71 when there is something physicaly wrong with either the disk drive or disk and the drive cannot read/write properly to the disk. You also get this error by breaking the program during such activity. Error 72 merely means that you've put the wrong disk number on the new file name during a NAME.

As with most errors, they can be used to your advantage. The technique is to force a possible error before it would ruin your program and then redirect the flow to handle the problem. For instance, suppose you want to know if a certain file (called "test") exists. You can do FILES and see directly however this requires the user to alter their choice after seeing the result and might disrupt your display. One way of avoiding this is to use:

```
NAME"test" AS "test"
```

This will produce either a File not found error (53) if the file is not on the disk or File already exists (67) if it is there. In the error trapping subroutine, you set up a sequence like:

```
IF ERR=53 then RESUME #### ELSE IF ERR=67 THEN RESUME @@@@
```

where #### is the starting linenumber for the routine that executes if the file doesn't exist, and @@@@ is the start of the subroutine executed when it does exist.

Disk I/O errors are the hardest errors to handle when they are due to hardware/firmware faults. On some cheap disks you may get what are called bad or corrupted sectors. These sectors are not reliable for use and the drive will hang, attempting to read/write to them. SETting a disk to read after write ("R") means that the system will check that what is in a sector matches with what is written to it. If the two don't match, another write is done until the two do match. A file SET to "R" will force the same procedure, but only for sectors belonging to that file.

On other systems, bad sectors can be "locked out" of a directory and for all intensive purposes don't exist. This loses data but saves your nerves. On the SV however, this can't be done on such a basis. If you have a track that has too many bad sectors, or has them in bad locations (like sector 1) then you can block out that track by giving it an FEH entry on the Files Allocation Table (FAT). This value is used to reserve both the system file and directory track. The track will then not be used for a File and the bad sectors avoided. If you have bad sectors on the directory track, the best bet is to copy what files you can to another disk and use the disk for CP/M, or another computer, or anything.

In the above case, copying might prove difficult. If LOAD & SAVE & COPY don't work properly because the DIR or FAT is corrupted, then you can use DSKO$ & DSKI$ to transfer a file on a sector by sector basis. This is a long and tedious affair. You will also have to do some detective work using either DISK SCANNER (see part 14) or similar utility to see exactly which tracks and sectors the file is contained in. The listing FILE STATS will list full details of all files on a normal disk. After a file is transferred (to a formatted empty disk) you then use DISK SCANNER (hereafter called DS for brevity) to modify both the DIR and FAT on the new disk. If you have been following the last two parts, you should have an idea how this is done.

Let's take an example of an imaginary bad disk and a file called "test". The FAT tables are beyond repair and we want to get a copy of test because it's the only copy you have and it took you 20 hours to design and type in.

First, you create a new formatted disk using the CP/M format program to format it and the basic format program to create a vacant directory track. Using DS you switch to ASC and MOD. mode to look at the DIR on track 20, sector 1. You find the entry for the program. The first six bytes show "test ". The next byte is 200 octal indicating it is a binary save and the next byte is set to 19, indicating that the first track used for the program is track 19. You go to track 19 and there is the start of "test". Checking along, one sector at a time, you discover that "test" uses all 17 sectors of the track and seems to end abruptly on the 17th. This implies that it is continued on another track.

After 10 minutes search you find what appears to be the rest of "test" on track 36. It occupies sectors 1 to 6. At this point you use DS to transfer the sectors from one disk to another (if you have two drives) or write a small program that uses DSKI$ to get a sector and then (after you have placed the new disk in the drive) uses DSKO$ to dump it to the new disk. It will transfer all sectors of track 19 and the first 6 sectors of track 36 from the first to the second disk. The listing TRACK COPIER will allow you to copy tracks and sectors between either two drives or even the same drive, with one or two disks. After the sectors have been dumped you use DS to alter the DIR. On the first entry you place "test "followed by 200 octal and 19. On sector 15 of track twenty you alter the FAT so that entry 19 reads 36 (it then points to entry 36) and on 36 you place 306 octal, indicating the end of file and the first six sectors are used. Sector 15 is then copied to sectors 16 and 17. If you exit DS and do a FILES you will see "test" listed. With luck, if you load "test" you will get your original program restored to you.

It doesn't always work because there can be some doubt as to where a file ends, but at least you recover the bulk of your program. The key to the procedure is the amount of detective work you do. You need to have a clear idea of what you are looking for before you can find it. If either the DIR or FAT is blown, you use the other to help find the file. If both are blown, you have to spend time looking at sector 1's of each track to see if it looks like the start of your file. Good luck!

NEXT MONTH

In next months part, I'll talk about menus, subroutines and interrupts.

TRACK

by : L.A. Dunning

This Program may be entered using the 'INPUT' program from Newsletter 2 - 2 (NOV. 84.) or The Year Book.

```
AC      10 REM   TRACK COPIER
DM      20 :
CC      30 SCREEN0,0:CLS:WIDTH39:PRINTTAB(13)"TRACK COPIER":CLEAR5000:DEFIN
           TA-Z:DEFSTRV:DIM V(17,1),VB(1):VE=CHR$(27):FIELD#0,128ASVB(0),12
           8ASVB(1)
KB      40 LOCATE0,1:INPUT"How many drives on line";ND:IFND>2ORND<1GOTO40
HO      50 LOCATE0,4,1:PRINT"SECTORS COPIED FROM":PRINT"------- ------ ----
           "
BN      60 LOCATE0,6:PRINTUSING"Disk           (1 or #) ";ND;:INPUTD1:IFD1<1
           ORD1>NDGOTO60
BL      70 LOCATE0,7:INPUT"Track          (0 to 39)";T1:IFT1<0ORT1>39GOTO70
FM      80 LOCATE0,8:INPUT"First Sector ( 1 to 17)";SA:IFSA<1ORSA>17GOTO80
DP      90 PRINT" to":SB=0
AO     100 LOCATE0,10:PRINTUSING"Last  Sector (## to 17)";SA;:INPUTSB:IFSB=
           0THENSB=SA
BG     110 IFSB<SA OR SB>17GOTO100
EC     120 LOCATE0,12,1:PRINT"SECTORS COPIED TO":PRINT"------- ------ --"
BH     130 LOCATE0,14:PRINTUSING"Disk           (1 or #) ";ND;:INPUTD2:IFD2<
           1ORD2>NDGOTO130
BF     140 LOCATE0,15:INPUT"Track          (0 to 39)";T2:IFT2<0ORT2>39GOTO70
BL     150 SC=0:SM=17-SB+SA
IO     160 LOCATE0,16:PRINTUSING"First Sector ( 1 to ##)";SM;:INPUTSC:IFSC=
           0THENSC=SA
BB     170 IFSC<1 OR SC>SMGOTO160
CK     180 SD=2:IFD1<>D2GOTO200
DI     190 LOCATE0,18,0:PRINT"Are you copying to the same disk <Y/N>":GOSUB
           270
DJ     200 LOCATE0,20,0:PRINT"Insert original disk in drive "D1:PRINT"Press
            <ESC> when ready":GOSUB290:NC=SB-SA:ON SD GOTO 220,210
AG     210 IFD1=D2GOTO240ELSEGOSUB280
DA     220 FORS=0TONC:V=DSKI$(D1,T1,SA+S):DSKO$D2,T2,SC+S:NEXT:GOTO250
DI     230 GOSUB280:FORS=0TONC:V=DSKI$(D1,T1,SA+S):FORE=0TO1:V(S,E)=VB(E):N
           EXT:NEXT
BH     240 GOSUB280:FORS=0TONC:FORE=0TO1:LSETVB(E)=V(S,E):NEXT:DSKO$D2,T2,S
           C+S:NEXT
EF     250 LOCATE0,18:PRINTVE"J";:PRINT"Do you want to copy more sectors <Y
           /N>":GOSUB270:ONSDGOTO260,300
KF     260 PRINT"Use the same original specs      <Y/N>":GOSUB270:LOCATE0,1
           8:PRINTVE"J":ONSDGOTO120,50
AI     270 V=INPUT$(1):SD=INSTR("YyNn",V)/2+.5:IFSD=0GOTO270ELSERETURN
HO     280 LOCATE0,22:PRINT"Insert copy     disk in drive "D2:PRINT"Press <
           ESC> when ready";
DH     290 V=INPUT$(1):IFV<>VEGOTO290ELSERETURN
CL     300 CLS:SCREEN0,1:LOCATE,,1
END
```

STATS

by : L.A. Dunning

This Program may be entered using the 'INPUT' program from Newsletter 2 - 2 (NOV. 84.) or The Year Book.

```
BI      10 REM  FILE STATS
FO      20 REM  Lists Name, Type, last sector          used and tracks used
            by file.
DN      30 :
FB      40 CLS:MAXFILES=2:CLEAR2000:DEFINTA-Z:DEFSTRC,S:DIM S(16),C(40),N(4
           0)
EF      50 A=1:B=0:D=0:X1=VARPTR(#0)+9
KF      60 FIELD#0,16ASS(0),16ASS(1),16ASS(2),16ASS(3),16ASS(4),16ASS(5),16
           ASS(6),16ASS(7),16ASS(8),16ASS(9),16ASS(10),16ASS(11),16ASS(12),
           16ASS(13),16ASS(14),16ASS(15)
FB      70 LOCATE0,10:PRINT"List FILES to <S>creen":PRINTTAB(11)"or <P>rint
           er"
AG      80 C=INPUT$(1):OP=INSTR("SsPp",C):IFOP=0GOTO80
MC      90 IFOP>2THENSF="LPT"ELSESF="CRT"
IK     100 OPENSF+":FILES"FOROUTPUTAS#1
GF     110 PRINT:PRINT#1,"Name       Type  Last,Tracks used":PRINT#1, "
                    Sctr
CE     120 D$=DSKI$(1,20,A)
CI     130 S(16)=S(B)
BK     140 TP=ASC(S(16)):IFTP=255GOTO260
BD     150 IFTP=0GOTO240
CK     160 D=D+1:CC=LEFT$(S(16),9)+" "
HE     170 TP=ASC(MID$(S(16),10)):FM=TPAND&O707:EX=TPAND&O70:IFFM=0THENCC=C
           C+"ASC"
CE     180 IFFM=1THENCC=CC+"MCR"
BE     190 IFFM=128THENCC=CC+"BIN"
BH     200 C1=" ":IFTPAND&O10THENC1="R"
BF     210 C2=" ":IFTPAND&O20THENC2="P"
BA     220 C3=" ":IFTPAND&O40THENC3="*"
DF     230 C(D)=CC+C1+C2+C3:N(D)=ASC(MID$(S(B),11))
CD     240 B=B+1:IFB>15THENB=0:A=A+1:GOTO120
AC     250 GOTO130
CB     260 IFD=0GOTO310
GM     270 D$=DSKI$(1,20,15):FORA=1TOD:CC="":J=N(A)
BJ     280 CC=CC+","+RIGHT$(STR$(J),2):J=PEEK(X1+J):IFJ<192GOTO280
AP     290 C(A)=C(A)+"("+RIGHT$(STR$(JAND63),2)+")"+CC:NEXT
HD     300 FORA=1TOD:PRINT#1,C(A):NEXT:GOTO320
BF     310 PRINT#1,"NO FILES"
AH     320 CLOSE
END
```

Cursor Alteration Program

By. A. Kellner (CP/M)

```
;************************ CURSOR ALTERATION PROGRAM ************************
;
;This  program  allows the user to alter the type and size  of  cursor
;when using
;the spectravideo 328 and the 80 column card.
;
;Written  by  A.  Kellner,  George Town,  Tas.        SVI & MSX  Users
;group.22/12/85
;
;***************************************************************************
;
        ORG     0100H   ;Base address of program
;
;************************ SYSTEM CONSTANTS ************************
;
BOOT    EQU     0000H   ;Reboot cp/m
BDOS    EQU     0005H   ;System call
CONIN   EQU     1       ;Console input function
CONOUT  EQU     2       ;Console output function
CONIO   EQU     6       ;Console i/o function
PSTR    EQU     9       ;Print string at console function
;
;************************ PROGRAM CONSTANTS ************************
;
CLS     EQU     0CH     ;Clear screen control code
CR      EQU     0DH     ;Carriage return control code
LF      EQU     0AH     ;Line feed control code
ESC     EQU     01BH    ;Escape control code
IVON    EQU     070H    ;Inverse video on [ p ]
IVOFF   EQU     071H    ;inverse video off [ q ]
;
CRTPT   EQU     050H    ;Address reg. select port [ 80 col card ]
DATAPT  EQU     051H    ;Control reg. data port      "
RTOP    EQU     0AH     ;Reg. for the top of the cursor
RBOT    EQU     0BH     ;Reg. for the bottom of the cursor
;
NOBL    EQU     00H     ;Value for a non blinking cursor
NOCS    EQU     020H    ;  "     "  no cursor
FASTBL  EQU     040H    ;  "     "  fast blinking cursor
SLOWBL  EQU     060H    ;  "     "  slow blinking cursor
;
;************************ SELECT TYPE OF CURSOR REQUIRED ******************
;
SELECT: LXI     D,MENU1
        CALL    PRINT   ;Print first menu
        CALL    INPUT   ;Get users response
        LDA     CIN     ;Load users input
        CPI     031H    ;test which option selected
        JZ      NBK     ;non-blinking
        CPI     032H
        JZ      NCS     ;no cursor
        CPI     033H
```

```
        JZ      SBK     ;slow blink
        CPI     034H
        JZ      FBK     ;fast blink
        JMP     SELECT  ;If not valid option loop
;
******************** SET TOP POSITION OF CURSOR ************************
;
SIZE:   LXI     D,MENU2
        CALL    PRINT   ;Print second menu
        CALL    INPUT   ;Get users response
        LDA     CIN
        CPI     030H    ;Test if input is in range
        JC      SIZE    ;if not loop
        CPI     039H
        JNC     SIZE    ;Not in range loop
        SUI     030H    ;Remove ascii offset
        STA     TOPSL   ;Store top scan line selected
        MOV     B,A
        LDA     CDAT    ;Load selected cursor type & add
        ADD     B       ;top scan line
        STA     CDAT    ;Store back in memory
;
;********************* SET BOTTOM POSITION OF CURSOR **********************
;
SIZE2:  LXI     D,MENU3
        CALL    PRINT   ;Print third menu
        CALL    INPUT   ; Same as above
        LDA     CIN
        CPI     030H
        JC      SIZE2
        CPI     039H
        JNC     SIZE2
        SUI     030H
        MOV     B,A
        LDA     TOPSL
        CMP     B       ;Check that bottom line no.
        JZ      STORE   ;is = or > top line no.
        JNC     SIZE2   ;If not loop
STORE:  MOV     A,B
        STA     BOTSL   ;Store bottom line no. in memory
;
;********************* SET CURSOR TO USER INPUT VALUES ********************
;
SETCRS: MVI     A,RTOP  ;load reg no. for top of cursor
        OUT     CRTPT   ;send to address port
        LDA     CDAT    ;load value to set
        OUT     DATAPT  ;send value to data port
        MVI     A,RBOT  ;load reg no. for bottom of cursor
        OUT     CRTPT   ;send to address port
        LDA     BOTSL   ;load value to set
        OUT     DATAPT  ;send value to data port
        JMP     SELECT  ;Go back to main menu.
```

```
;
;************* VALUES FOR TYPES OF CURSOR SUB-ROUTINES ******************
;
NBK:    MVI     A,NOBL  ;Non-blinking
        STA     CDAT
        JMP     SIZE
;
NCS:    MVI     A,NOCS  ;No cursor
        STA     CDAT
        JMP     SETCRS
;
SBK:    MVI     A,SLOWBL;Slow blink
        STA     CDAT
        JMP     SIZE
;
FBK:    MVI     A,FASTBL;Fast blink
        STA     CDAT
        JMP     SIZE
;
;*********************  UTILITY SUBROUTINES  ***************************
;
;**** EXIT ROUTINE ****
;
EXIT:   LXI     D,CLEAR ;Clear the screen
        CALL    PRINT
        JMP     BOOT    ;Go back to cp/m
;
;**** PRINT STRING AT CONSOLE ROUTINE ****
;
PRINT:  PUSH    H
        PUSH    B
        MVI     C,PSTR  ;Print string function
        CALL    BDOS
        POP     B
        POP     H
        RET
;
;**** PRINT SINGLE CHARACTER AT CONSOLE ROUTINE ****
;
PCHR:   MVI     C,CONOUT;Console output function
        CALL    BDOS
        RET
;
;**** GET CONSOLE INPUT ROUTINE ****
;
INPUT:  MVI     C,CONIO ;Console input function
        MVI     E,0FFH
        CALL    BDOS
        ORA     A       ;Test for input
        JZ      INPUT   ;If none loop
        CPI     ESC     ;If escape code
        JZ      EXIT    ;return to cp/m
        STA     CIN     ;Store input in memory
        MOV     E,A
        CALL    PCHR    ;Print input at console
        RET
```

```
;
;********************** STRING CONSTANT *********************
;
CLEAR:  DB      CLS,CR,LF,'$';String to clear the screen
;
;******************** SELECTION MENU ***************************
;
MENU1:  DB      CLS,LF,CR
        DB      '                  ========================== ',CR,LF
        DB      '                  CHANGE CURSOR SHAPE PROGRAM',CR,LF
        DB      '                  ========================== '
        DB      CR,LF,LF
        DB      '            ',ESC,IVON,'SELECT TYPE OF CURSOR REQUIRED'
        DB      ESC,IVOFF,CR,LF,LF
        DB      '   1)     - Non - blinking cursor ',CR,LF
        DB      '   2)     - No cursor ',CR,LF
        DB      '   3)     - Slow blinking cursor ',CR,LF
        DB      '   4)     - Fast blinking cursor ',CR,LF
        DB      '   ESC)   - To exit program ',CR,LF,LF
        DB      '          '
        DB      ESC,IVON,'ENTER YOUR SELECTION :',ESC,IVOFF,' $'
;
;********************* SELECT TOP OF CURSOR MENU ********************
;
MENU2:  DB      CLS,LF,CR
        DB      '                    SELECT SIZE OF CURSOR REQUIRED',CR,LF
        DB      '                    ------------------------------',CR,LF,LF
        DB      '            ',ESC,IVON,'TOP SCAN LINE POSITION',ESC,IVOFF
        DB      CR,LF,LF
        DB      'Enter [ 0 ] to start cursor at top of block ',CR,LF
        DB      'to [ 8 ] to start at the bottom or ESC to exit.',CR,LF
        DB      'E.G. set top & bottom scan line to 8 for a thin',CR,LF
        DB      'underline or top to 0 and bottom to 8 for the usual',CR,LF
        DB      'large block type of cursor. ',CR,LF,LF
        DB      '          '
        DB      ESC,IVON,'ENTER A NUMBER 0 TO 8 :',ESC,IVOFF,' $'
;
;******************** SELECT BOTTOM OF CURSOR MENU ********************
;
MENU3:  DB      CR,LF,LF
        DB      '          ',ESC,IVON,'BOTTOM SCAN LINE POSITION',ESC,IVOFF
        DB      CR,LF,LF
        DB      'Enter [ 0 ] to finish cursor at top of block ',CR,LF
        DB      'to [ 8 ] to finish at the bottom or ESC to exit.',CR,LF
        DB      'Note: The bottom scan line value must be equal to',CR,LF
        DB      'or greater than the top scan line value',CR,LF,LF
        DB      '          '
        DB      ESC,IVON,'ENTER A NUMBER 0 TO 8 :',ESC,IVOFF,' $'

;************************** DATA STORAGE AREA ***********************
;
CIN:    DS      1       ;Storage area for:- console input
CDAT:   DS      1       ;value for type of cursor
TOPSL:  DS      1       ;  "     "  top of cursor
BOTSL:  DS      1       ;  "     "  bottom of cursor
;
;================================================================================
;
        END
```

MSXSWEEP.BAS" (m.s.x.)

by : S.V. Mc Namee.

This Program may be entered using the 'INPUT' program from Newsletter 2 - 2 (NOV. 84.) or The Year Book.

```
CI      10 '**********************************
AO      12 '**** PROGRAM: MSXSWEEP.BAS *****
CE      14 '**********************************
CC      16 '
CA      18 ' AUTHOR: Steve McNamee
AC      20 ' DATE  : 16-2-86
CJ      22 ' USE   : This program is a com-
AP      24 ' prehensive file manipulation
BC      26 ' utility. It is designed to
FC      28 ' implement most of the features
DO      30 ' of the CP/M program NSWP2.COM
BD      32 ' The features not supported are
BE      34 ' the file print, file status
BJ      36 ' and squeeze/unsqueeze options.
CC      38 '
CG      40 ' IMPORTANT VARIABLES:
FP      42 ' NM$() - holds the file names
AG      44 ' AT()  - holds the attributes
AK      46 ' for each corresponding file
BK      48 ' name. Ø=untagged, 1=tagged,
JO      50 ' 3=file copied...possible retag
EC      52 ' and 3=deleted
DG      54 ' LN()  - holds the length in kBytes
BM      56 ' of each corresponding file name
DF      58 ' CF    - is the current file poin-
FE      60 ' ter
FJ      62 ' TL    - is the total length in k
GD      64 ' bytes of all tagged files
BK      66 ' CM    - is the maximum file number
AE      68 ' DN$   - is the logged drive name
CB      70 ' DN    - is the logged drive number
CM      72 '
EK      74 ' NOTE: This program has only been
BL      76 '       tested on a SV-738 X-Press.
BG      78 '       It should work OK on any MSX
BL      80 '       machine that has at least
BI      82 '       32k but this cannot be guar-
AG      84 '       anteed.
CJ      86 '
CH      88 '
DA      90 '
CC     100 CLEAR 4ØØØ,&HDØØØ
DE     110 ONSTOPGOSUB 9900:STOP ON
CP     120 GOSUB9000 'load machine code into mmory
DI     130 DEFINT A,L,C,N :DIMNM$(256),LN(256),AT(256)
BE     135 WIDTH40:LOCATE,,1
CP     140 DN$="A:":DN=1:POKE&HDØA0,DN 'set drive number in FCB
BC     200 'SEARCH FOR FIRST &HDØØØ
EK     210 'SEARCH FOR NEXT  &HDØ2Ø
CN     215 'COMPUTE FILE LENGTH &HDØ9Ø
AE     220 DEFUSR=&HDØØØ:DEFUSR1=&HDØ2Ø:DEFUSR2=&HDØ9Ø
```